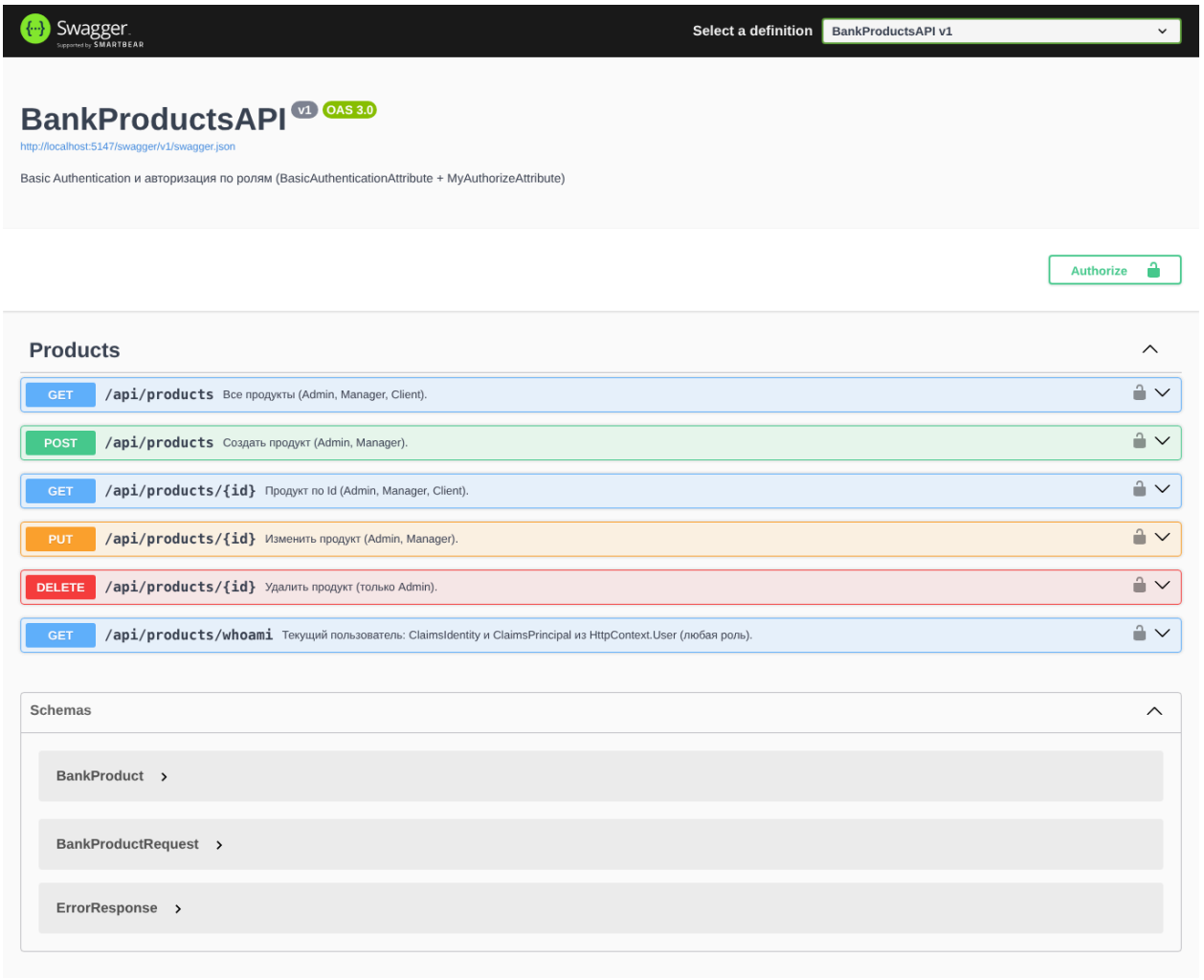
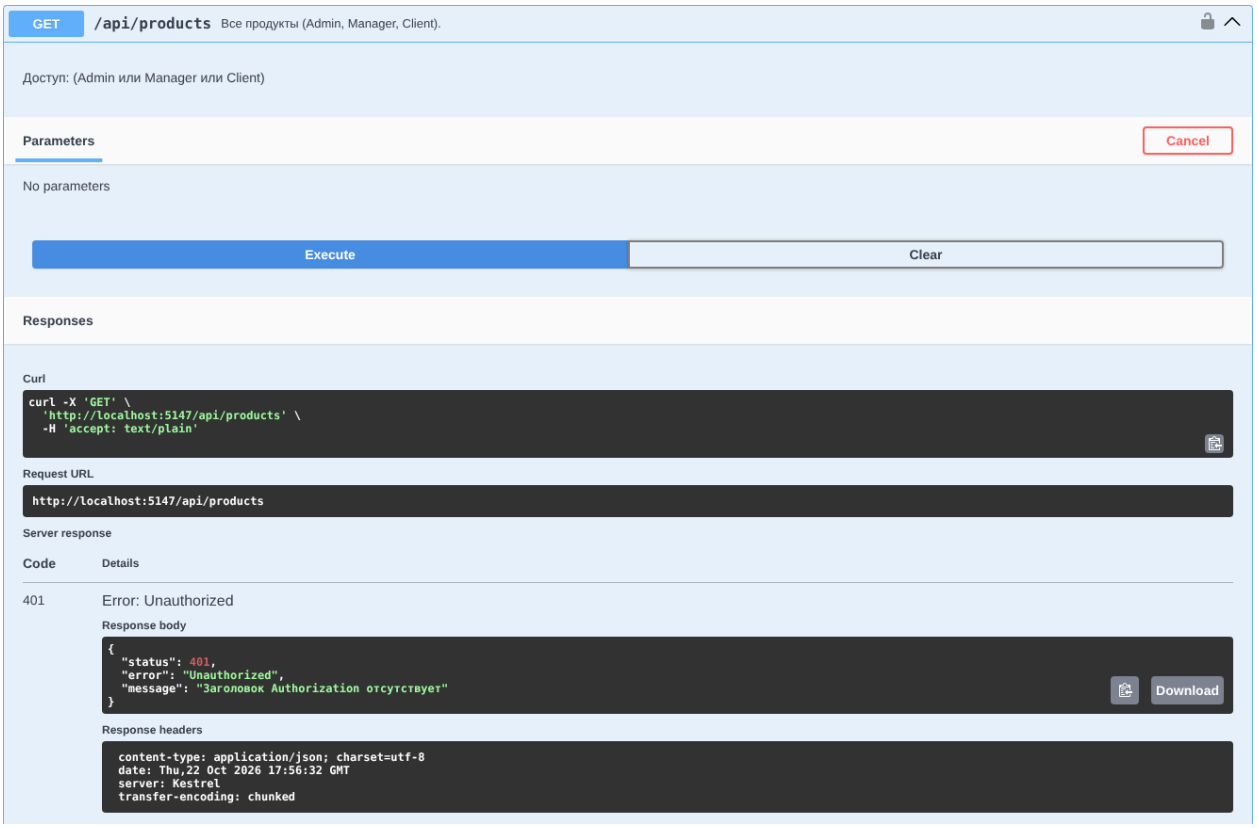
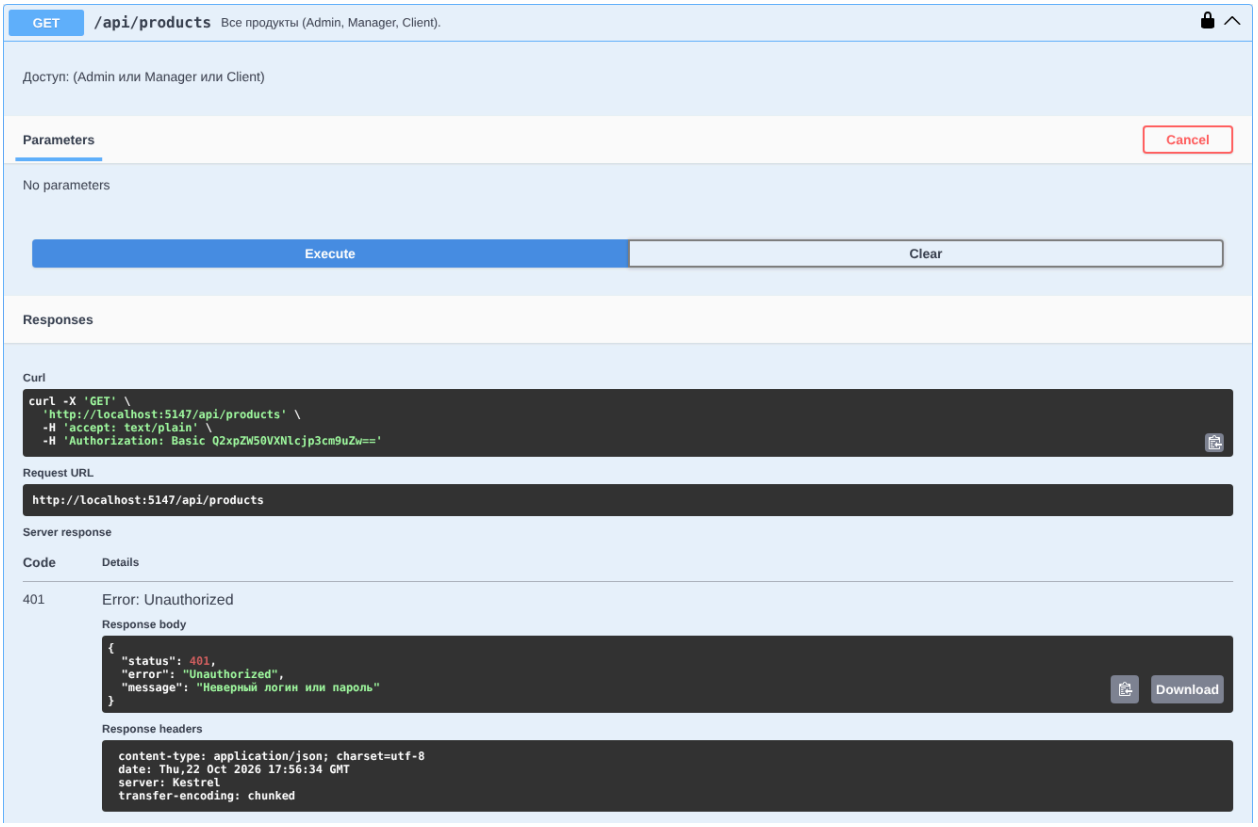
Лабораторная работа 7



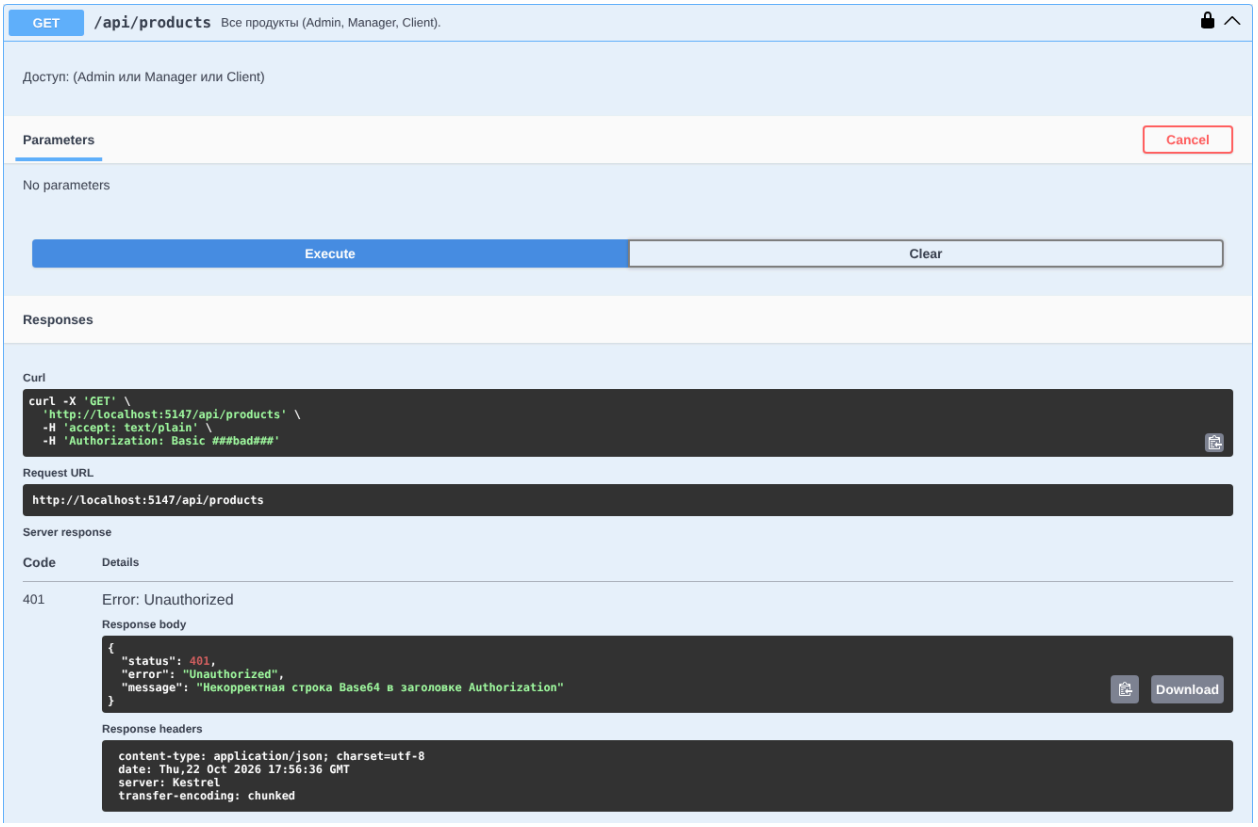
1 - Swagger UI BankProductsAPI: все методы требуют Basic Authentication



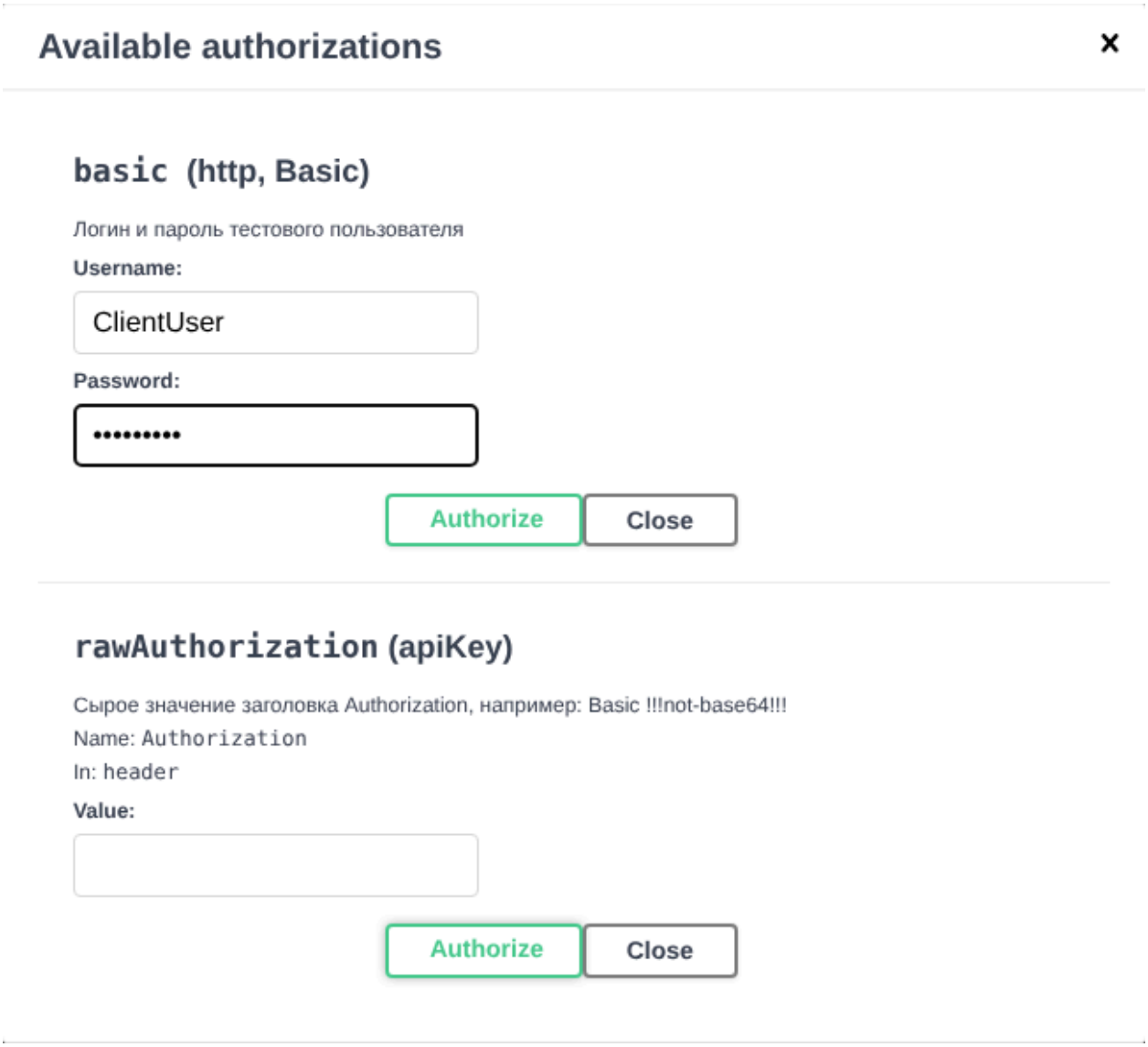
2 - GET /api/products без учётных данных - 401 Unauthorized



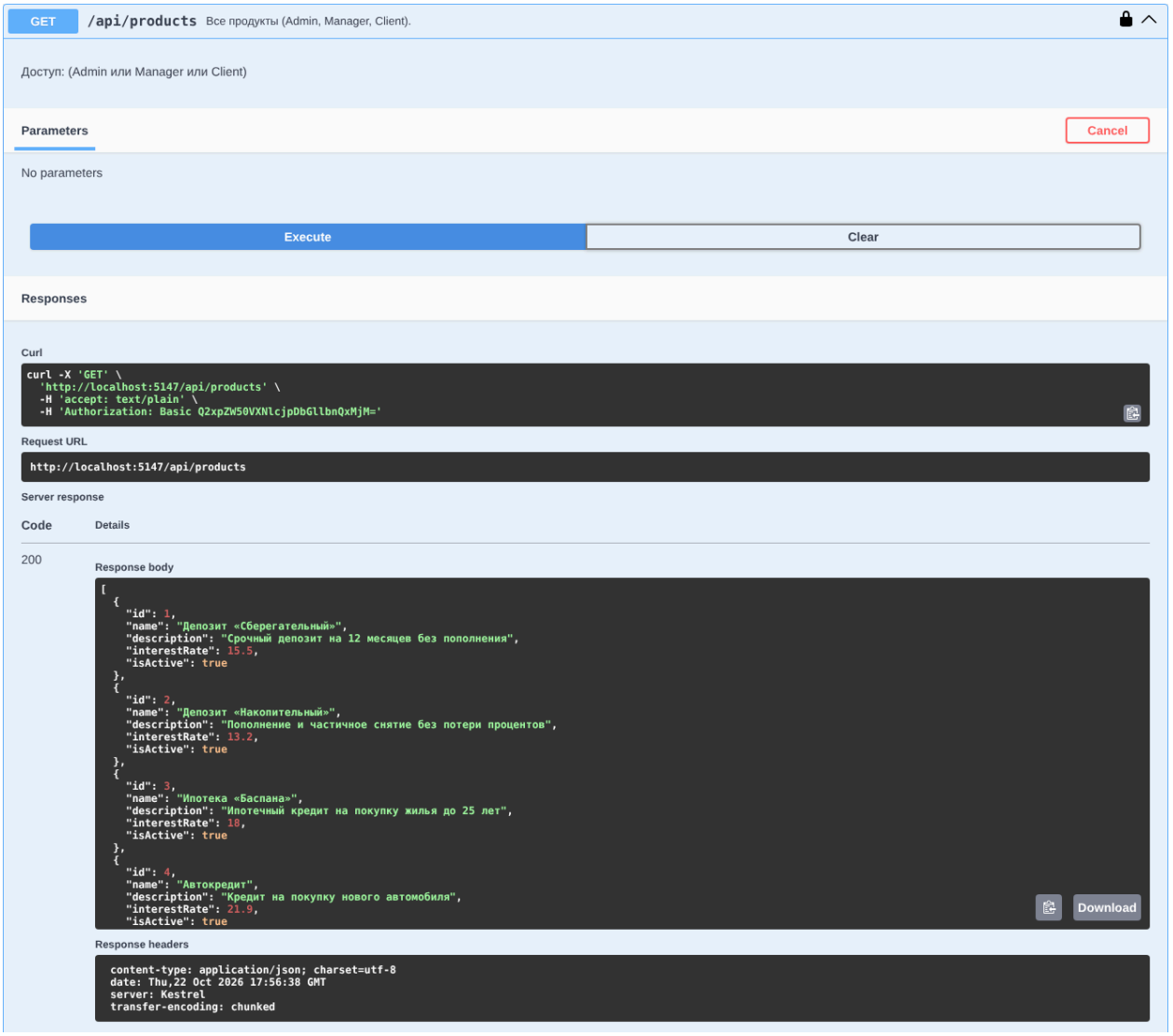
3 - Неверный пароль ClientUser - 401 Unauthorized



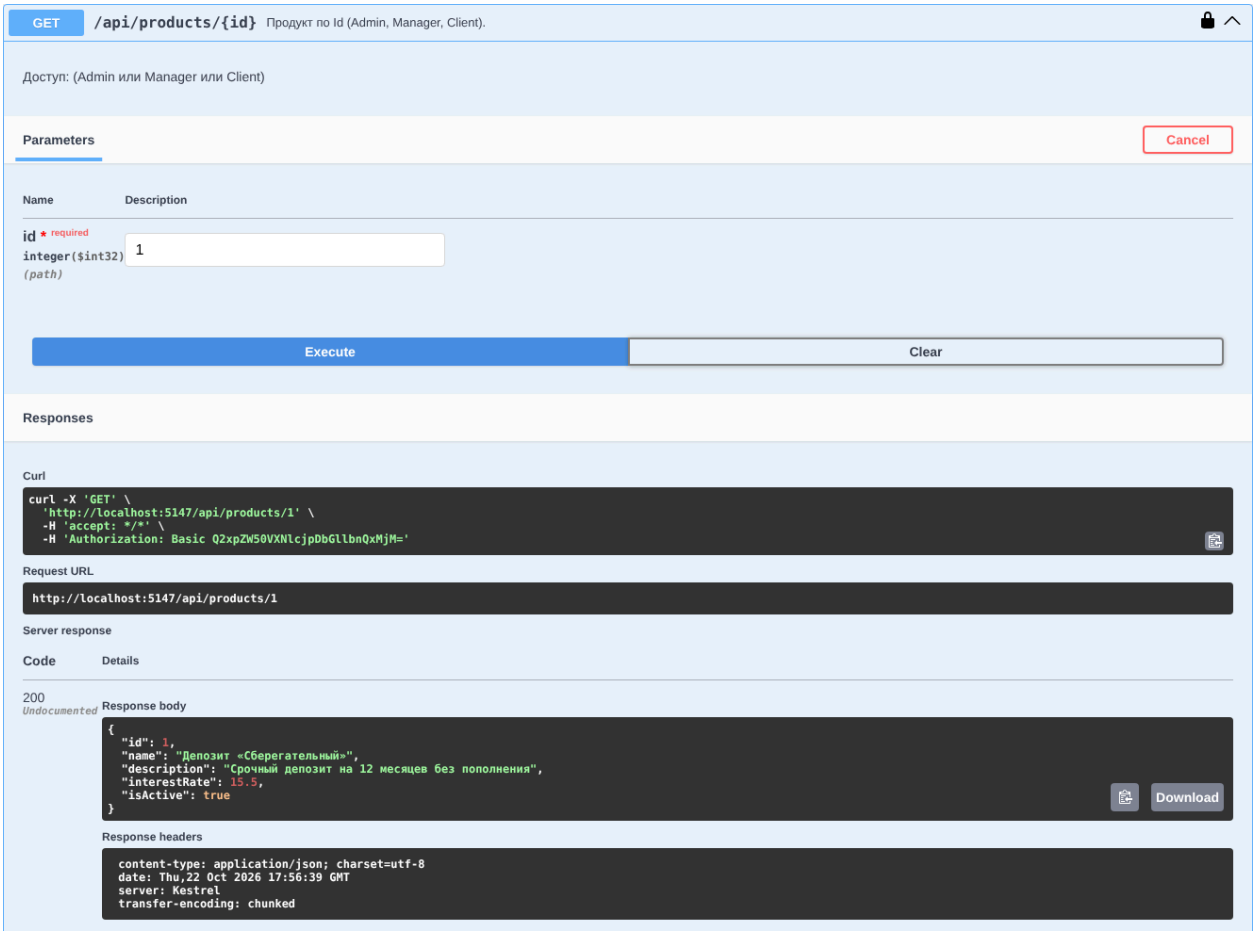
4 - Некорректная строка Base64 - 401 Unauthorized (FormatException обработано)



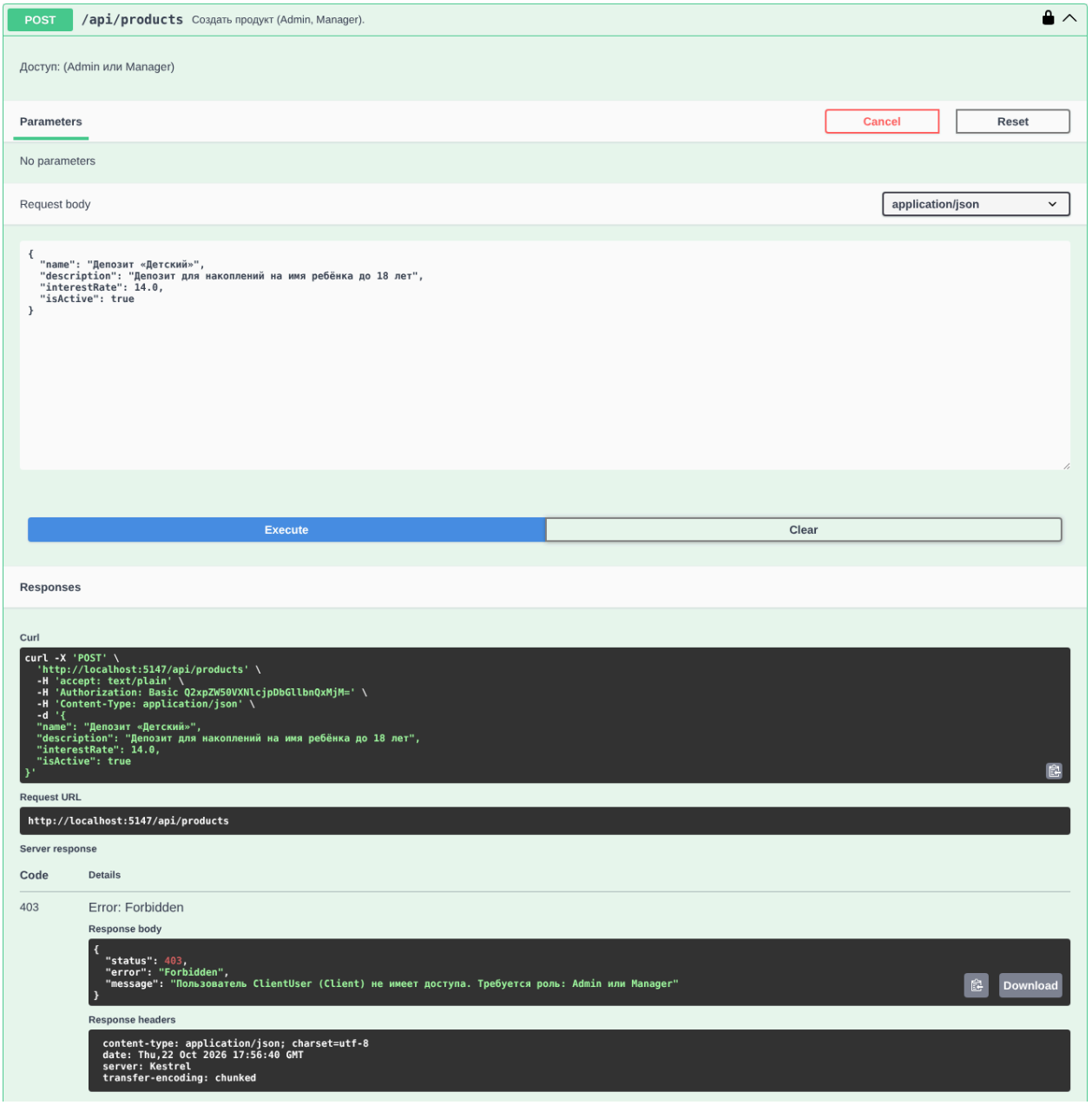
5 - Окно Authorize: ClientUser / Client123



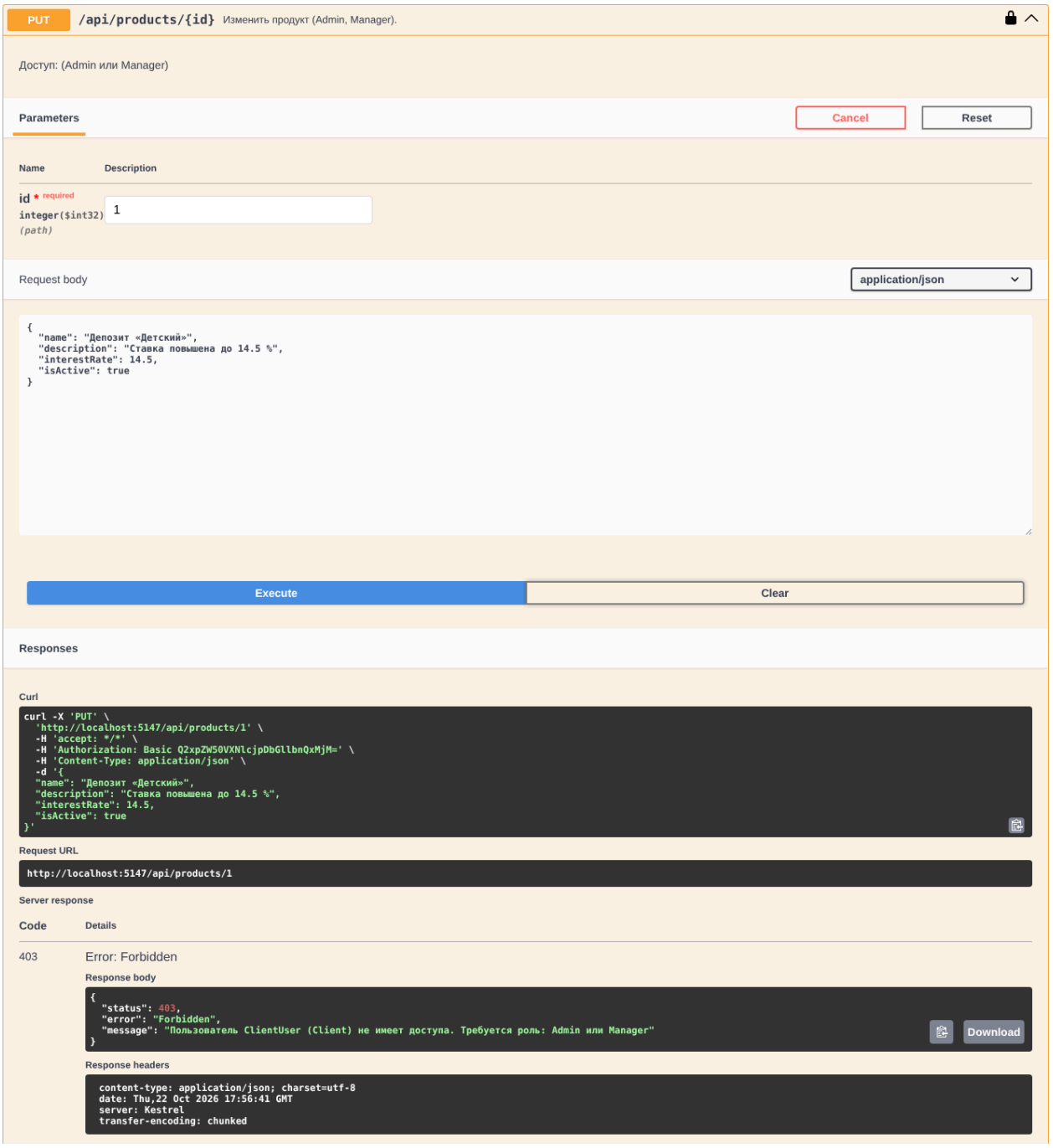
6 - Client: GET /api/products - 200 OK, 6 продуктов



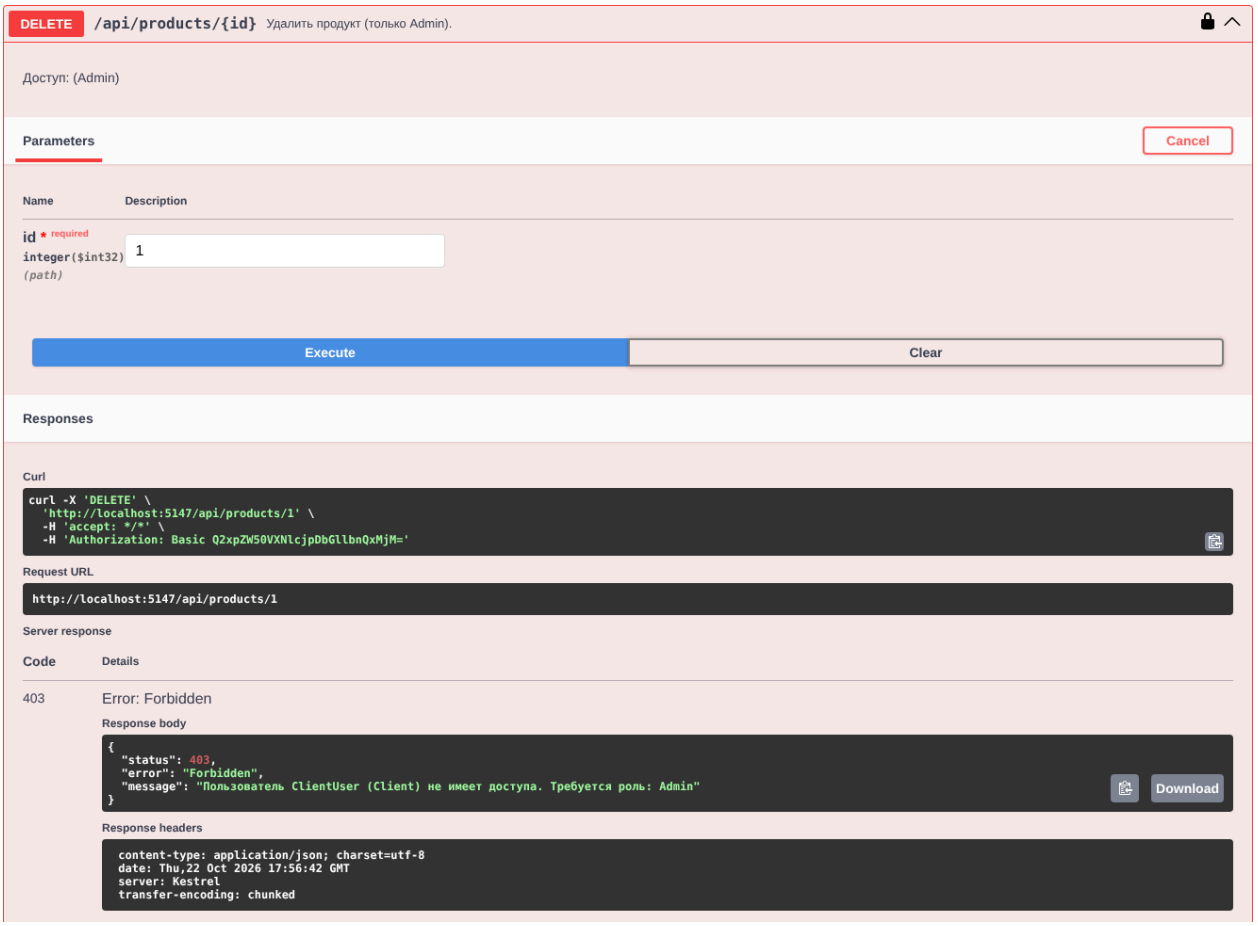
7 - Client: GET /api/products/1 - 200 OK



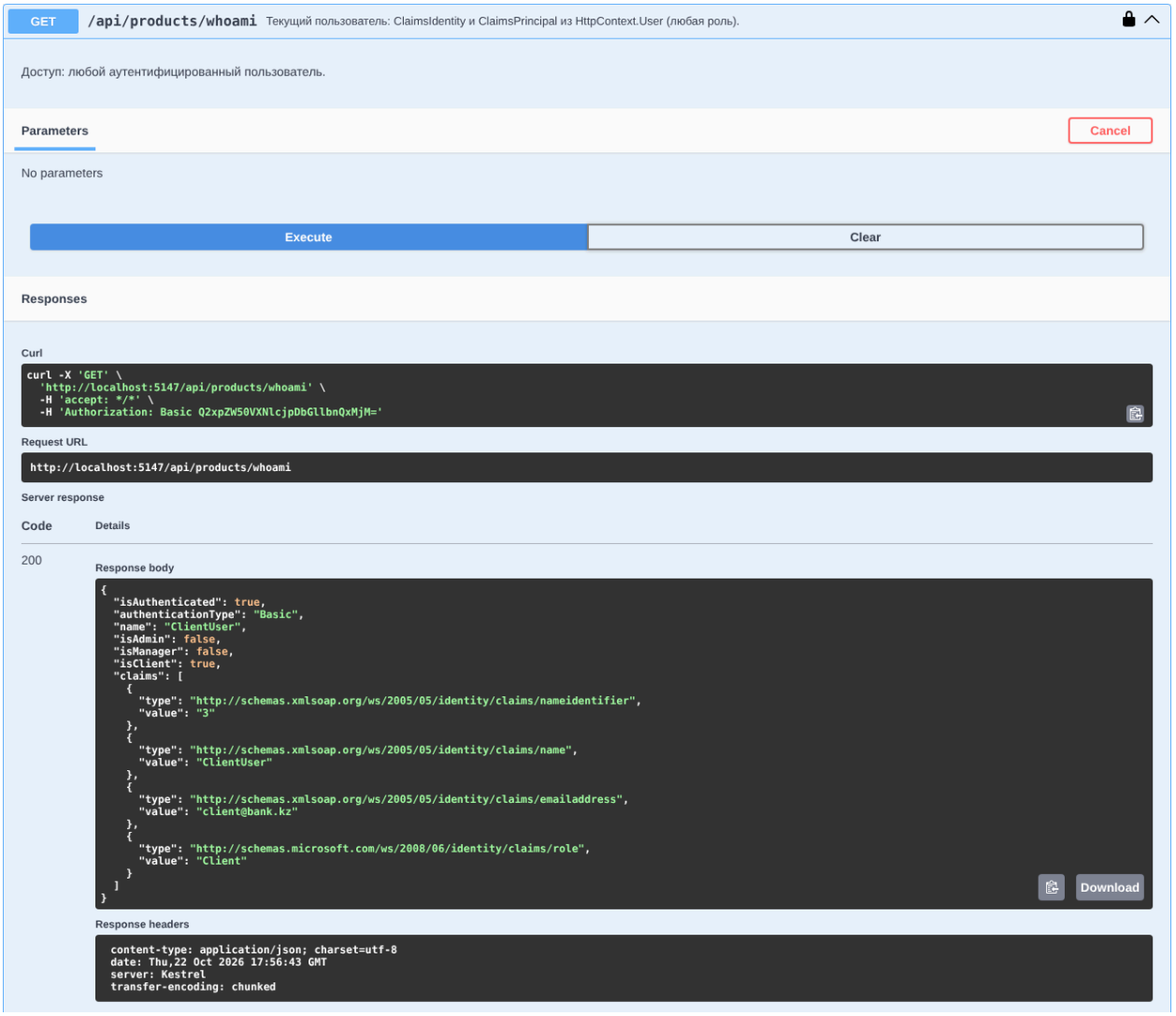
8 - Client: POST /api/products - 403 Forbidden (только просмотр)



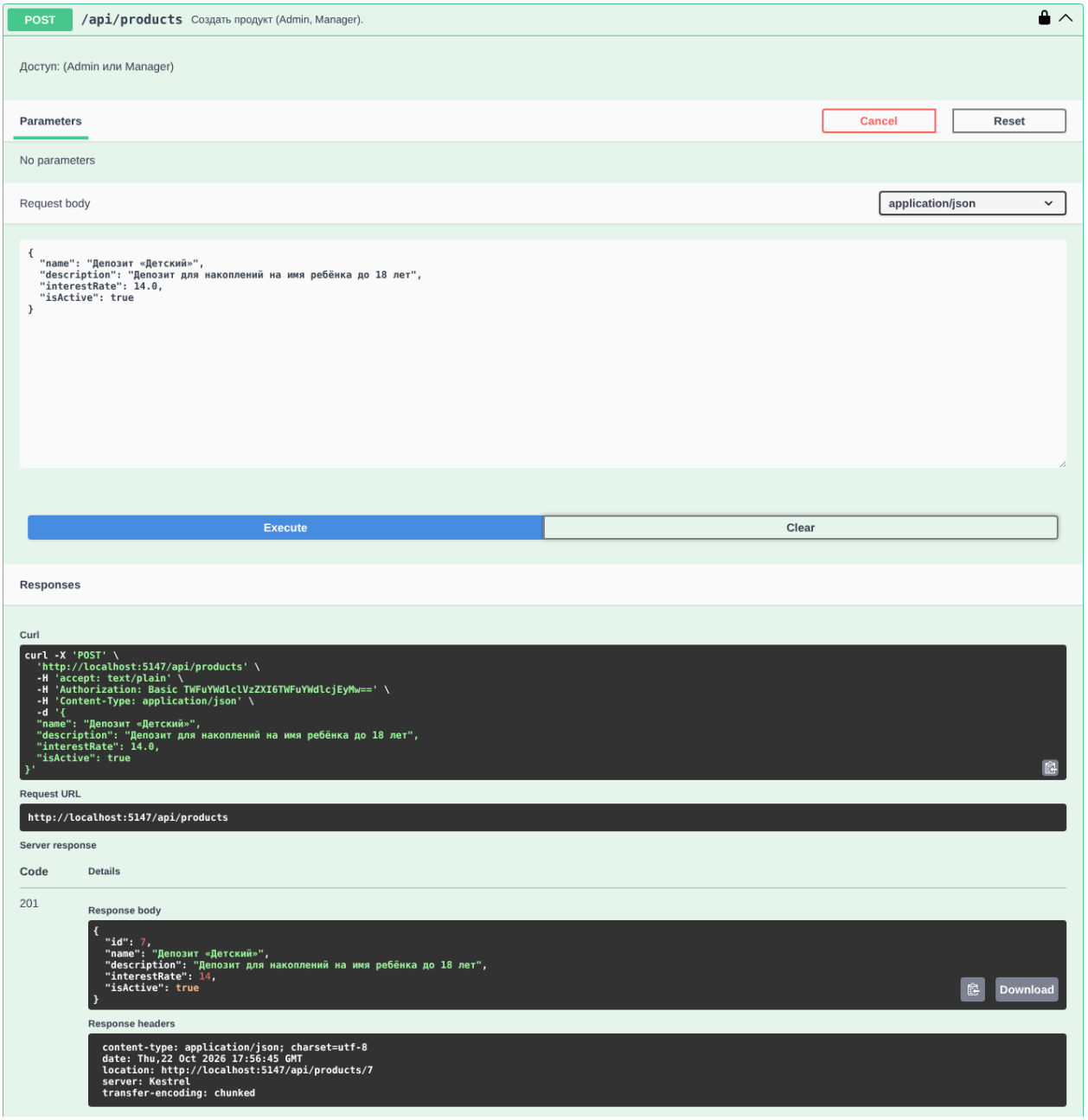
9 - Client: PUT /api/products/1 - 403 Forbidden



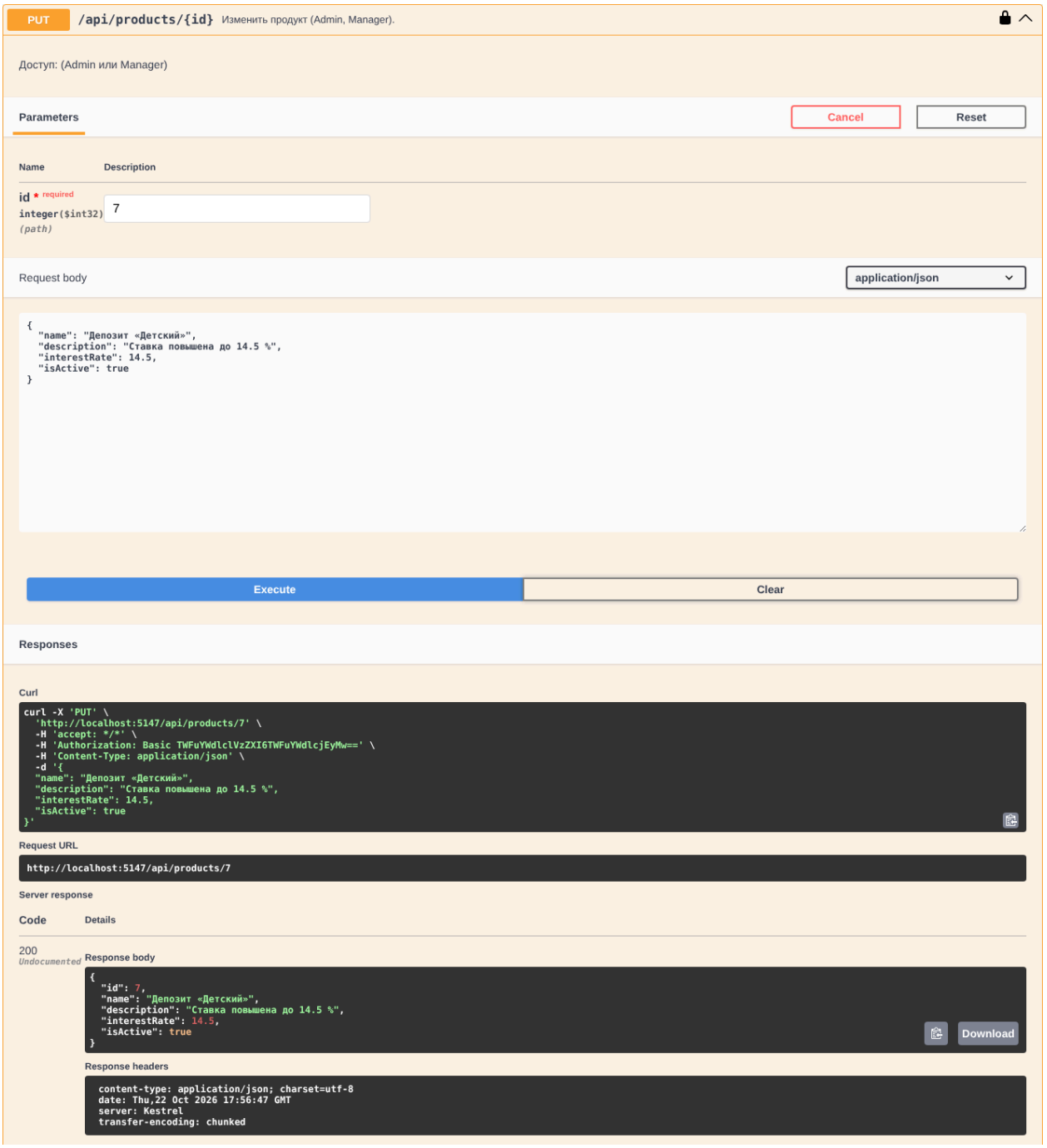
10 - Client: DELETE /api/products/1 - 403 Forbidden



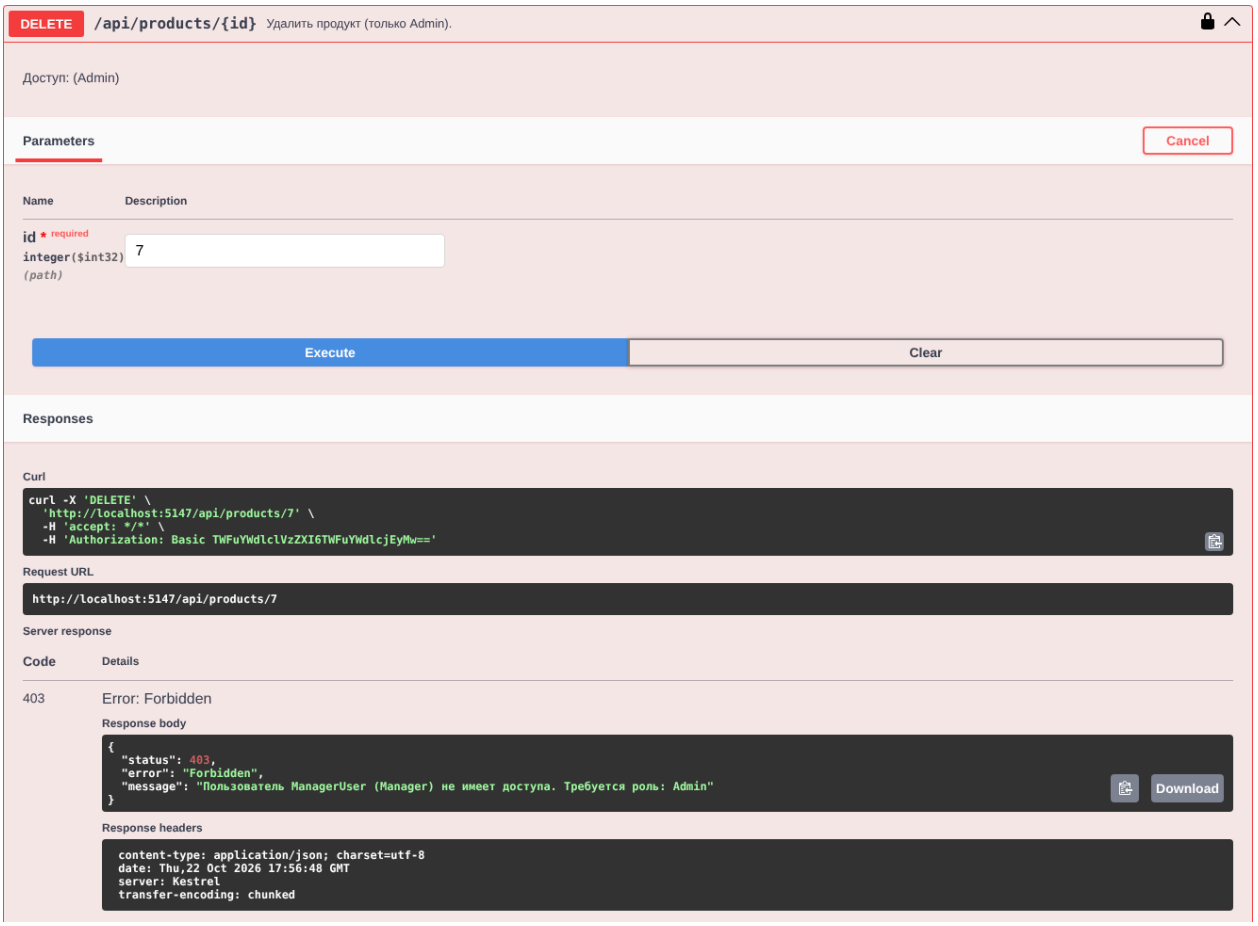
11 - Client: whoami - ClaimsPrincipal, IsInRole("Client") = true



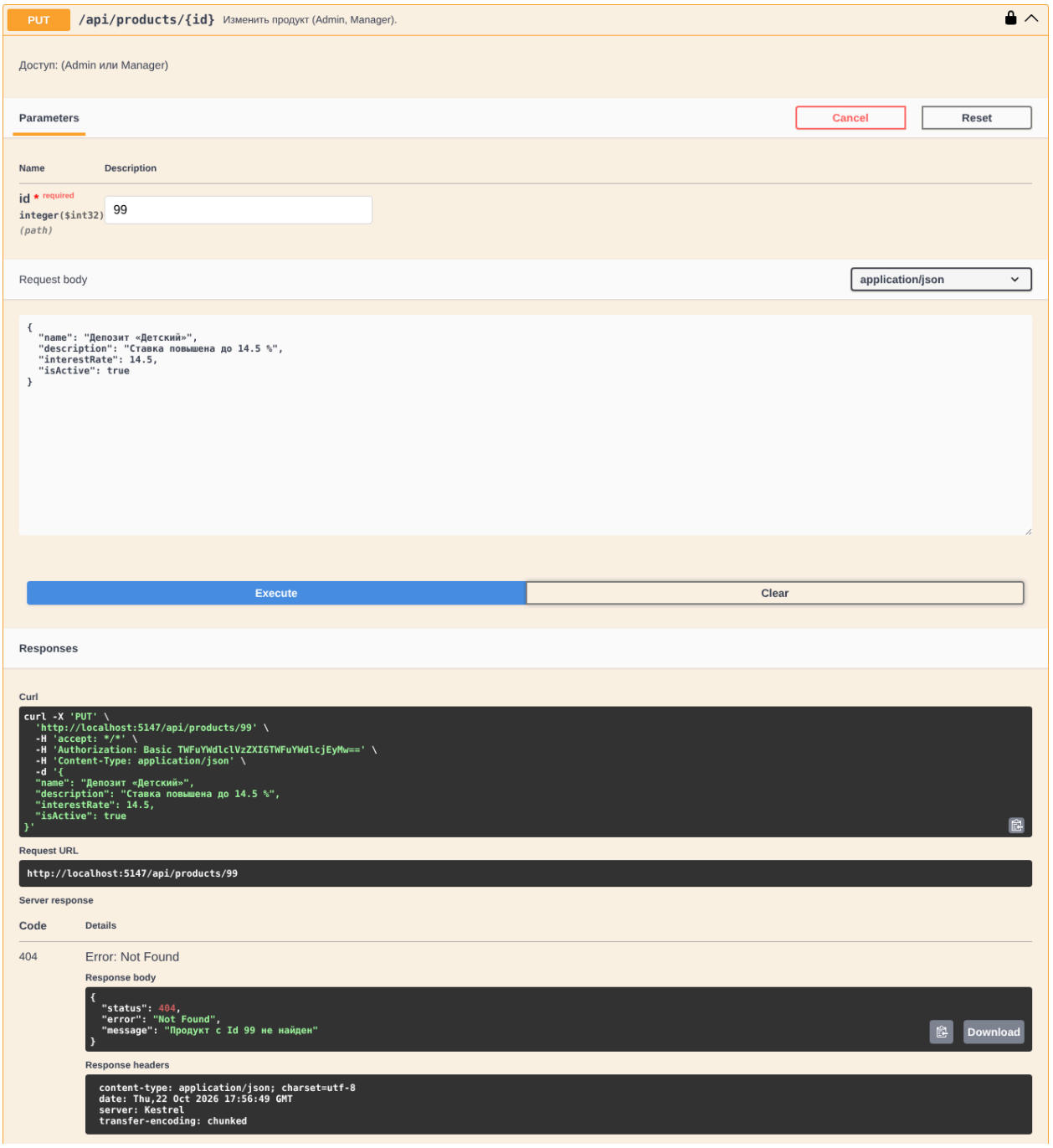
12 - Manager: POST /api/products - 201 Created



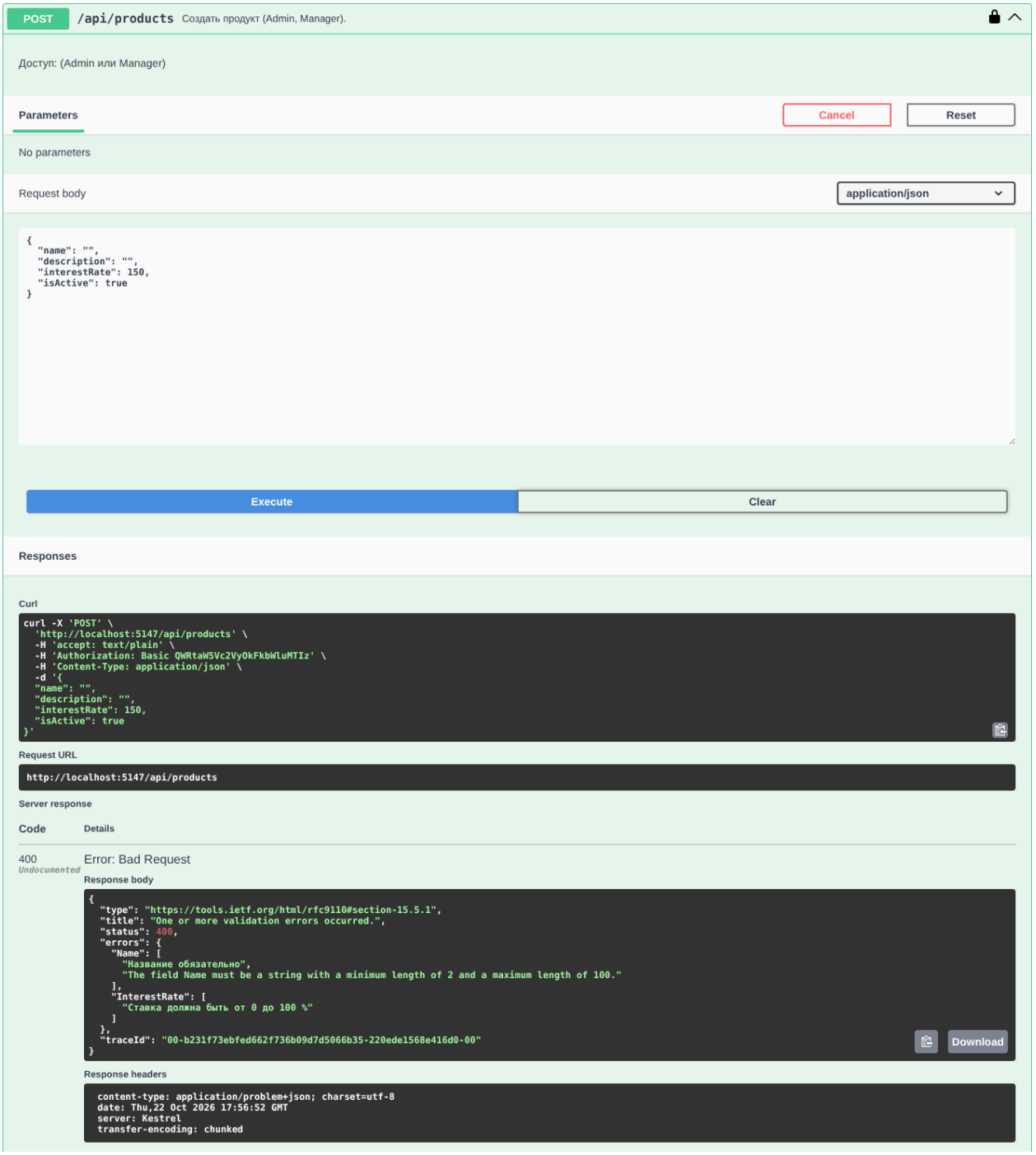
13 - Manager: PUT /api/products/7 - 200 OK



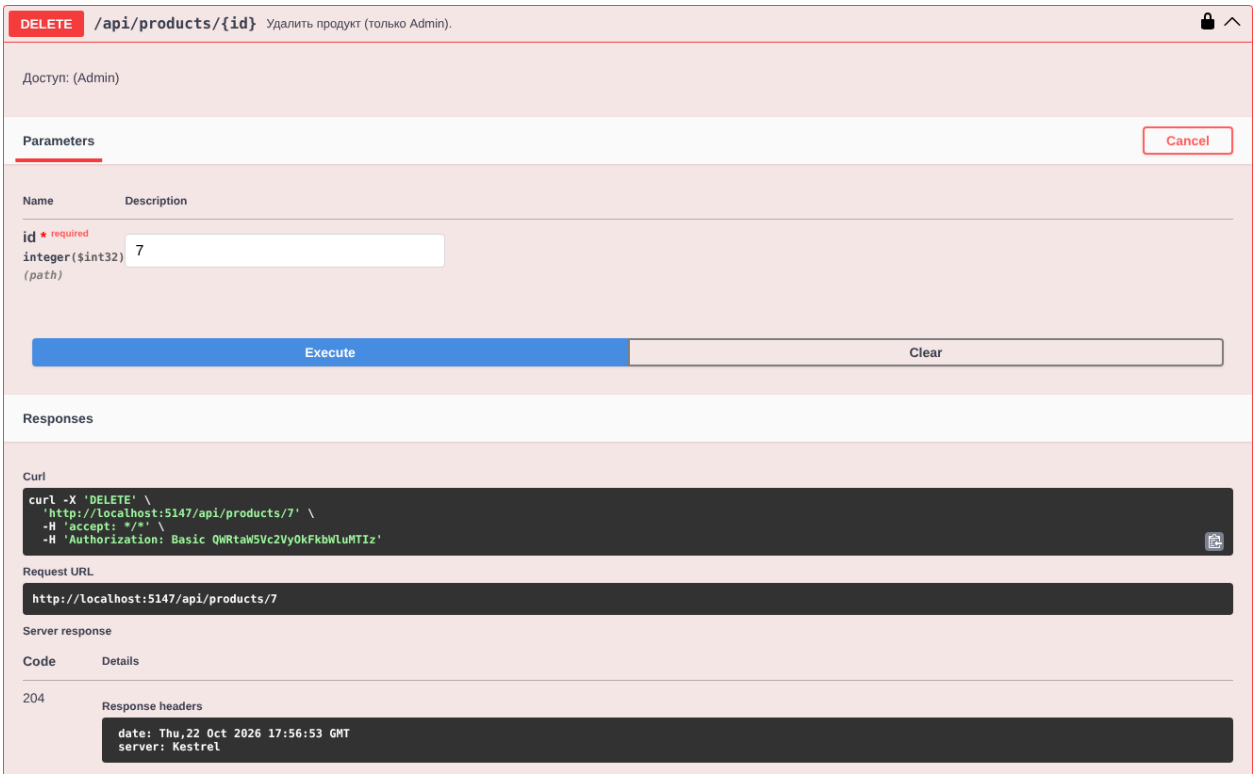
14 - Manager: DELETE /api/products/7 - 403 Forbidden (удаление только Admin)



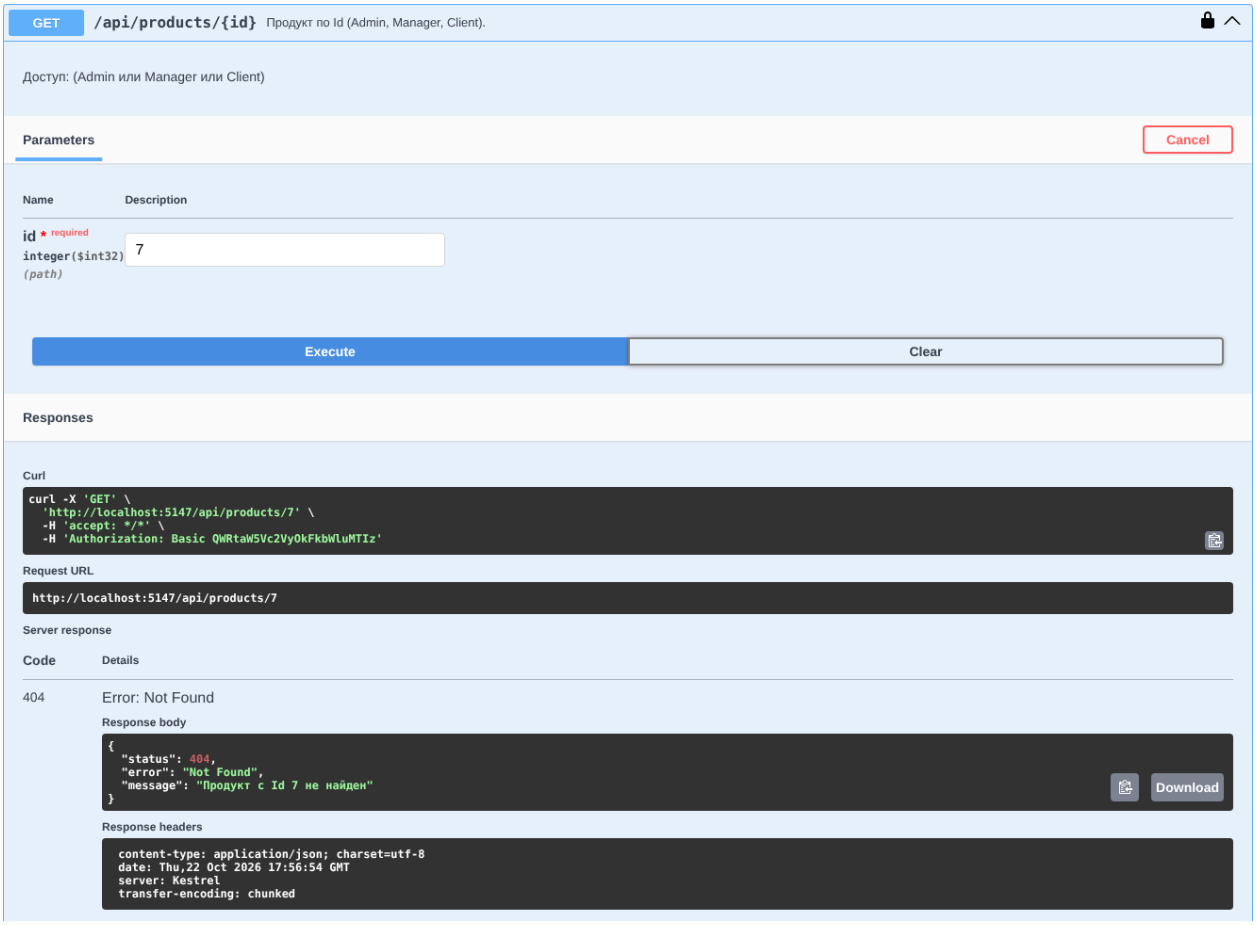
15 - Manager: PUT /api/products/99 - 404 Not Found



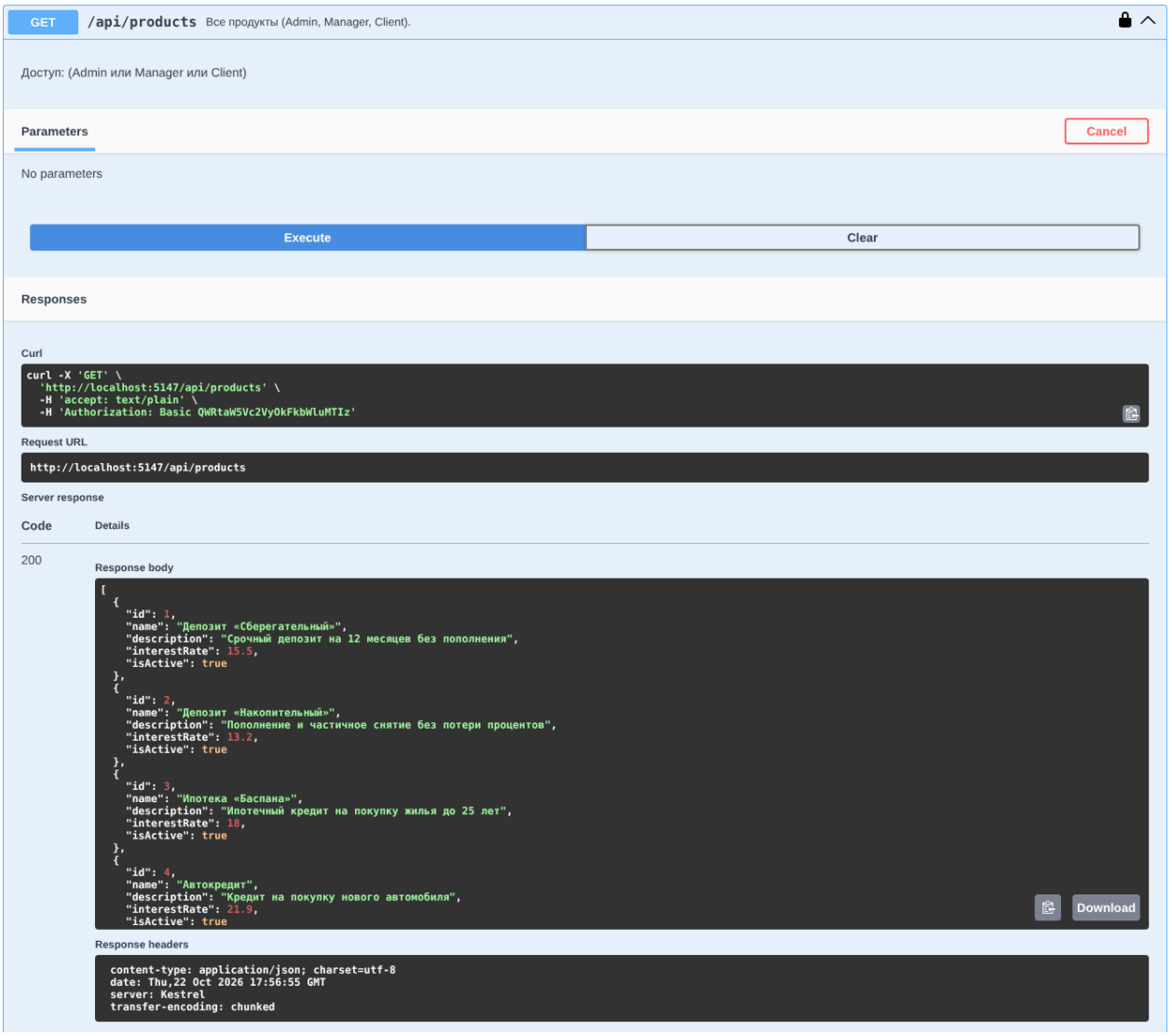
16 - Admin: POST с некорректными данными - 400 Bad Request (валидация)



17 - Admin: DELETE /api/products/7 - 204 No Content



18 - Admin: GET /api/products/7 после удаления - 404 Not Found



19 - Admin: GET /api/products - 200 OK

Контрольные вопросы

1. Что такое аутентификация?

Проверка подлинности пользователя: доказательство того, что он тот, за кого себя выдаёт. В BankProductsAPI сверка логина и пароля из заголовка Authorization с `UserService`; результат `ClaimsPrincipal` в `HttpContext.User`.

2. Что такое авторизация?

Проверка прав уже аутентифицированного пользователя на конкретное действие. В работе `MyAuthorizeAttribute`: например, ManagerUser аутентифицирован, но удалить продукт не может - 403 (рисунок 14).

3. Как работает Basic Authentication?

Клиент склеивает `логин:пароль`, кодирует в Base64 и отправляет в каждом запросе: `Authorization: Basic QWRtaW5Vc2VyOkFkbWluMTIz`. Сервер декодирует строку, делит по первому двоеточию и проверяет учётные данные. Без них или при ошибке сервер отвечает

401 с заголовком `WWW-Authenticate: Basic realm="..."`, приглашая клиента повторить запрос с логином и паролем. Состояния сессии нет — данные приходят в каждом запросе.

4. Для чего используется HTTP-заголовок Authorization?

Для передачи учётных данных серверу в формате `<схема> <данные>` (Basic, Bearer, Digest). Атрибут сначала проверяет наличие заголовка, затем схему, и только потом разбирает данные.

5. Какие сведения содержит ClaimsIdentity?

Список Claims (тип + значение): `NameIdentifier` = 2, `Name` = ManagerUser, `Email` = manager@bank.kz, `Role` = Manager; тип аутентификации `AuthenticationType = "Basic"`, из которого следует `IsAuthenticated = true`; свойство `Name` и типы Claim для имени и роли (`NameClaimType`, `RoleClaimType`). Пример - ответ `whoami` (рисунок 11).

6. Для чего необходим ClaimsPrincipal?

Это объект «текущего пользователя» в ASP.NET Core (`HttpContext.User`, `User` в контроллере). Он объединяет одно или несколько `ClaimsIdentity` и даёт методы `IsInRole`, `FindFirst`, `HasClaim`, по которым фильтры и контроллеры принимают решения.

7. Как работает метод IsInRole()?

Ищет во всех удостоверениях пользователя Claim с типом роли (`ClaimTypes.Role`) и указанным значением; сравнение без учёта регистра. `MyAuthorizeAttribute` вызывает его для каждой разрешённой роли: `Roles.Any(user.IsInRole)`.

8. В чём различие между HTTP 401 и HTTP 403?

401 - пользователь не аутентифицирован (нет или неверные учётные данные), запрос можно повторить с правильным логином. 403 - пользователь известен, но действие ему запрещено; повтор не поможет. Пример: Client без пароля - 401, Client с паролем при POST - 403.

9. Для чего используется интерфейс IAuthorizationFilter?

Для логики, которая должна выполниться до остальных фильтров и метода контроллера. `OnAuthorization` получает `AuthorizationFilterContext`; установка `context.Result` прерывает обработку. На нём построены оба атрибута; порядок задан через `IOrderedFilter`.

10. Какие риски безопасности существуют при использовании Basic Authentication?

Пароль передаётся в каждом запросе в Base64 — без HTTPS его легко перехватить. Нет срока действия и выхода из системы: украденный пароль работает, пока его не сменят. Нет встроенной защиты от подбора пароля (нужны блокировка и ограничение частоты). Браузер кэширует учётные данные. Пароли в этой работе хранятся открытым текстом только как учебный пример — в реальной системе нужны хеши (PBKDF2, bcrypt) и HTTPS.